Challenges with docker-compose: SPF (Not suitable for production environment) - Failover*

How to assign dynamic ports of the container to the LB.

Every container has **its own unique IP** address and every container get its **individual port number, in order to do load balancing** it's a challenge that whenever we redeploy the containers the port number will get changed.

It is helping us to make a multiple container from the same image, if we create 4 containers then each container gets **individual port number**, if we want to send a request to all containers then we must have a load balancer but what happens if we restart my container? The port number gets changed? How we manage this dynamic nature of the containers?

Compose utility is creating multiple containers in the same node, what happens if my node goes down?

We are creating multiple containers for HA, but what will happen if my node goes down? Will I get the HA? No, its SPF.

It's helpful in development but not recommended for production environment.

It provides feature of multiple replicas and scale up and scale down only.

How to manage dynamic nature of container (port change)?

Docker Swarm

How to run multiple containers from single/multiple images, into multiple VMs with a single command?

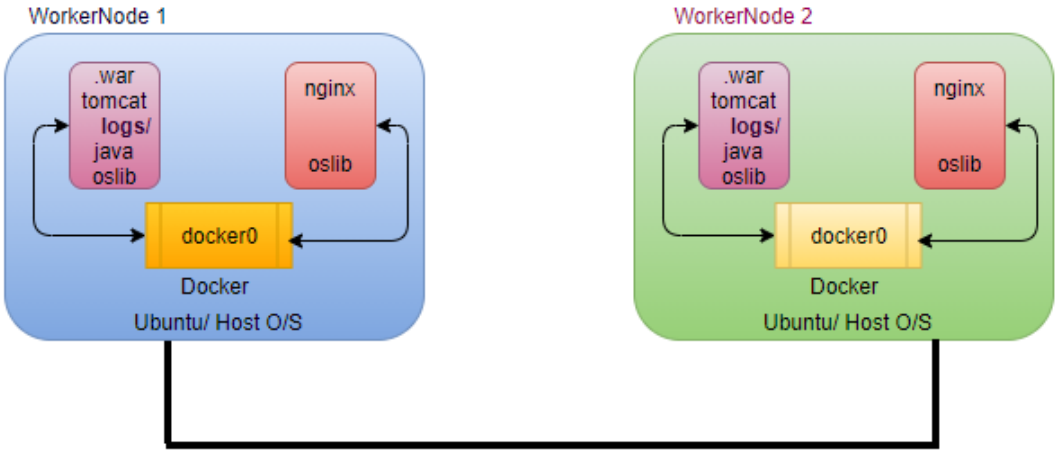
In production env maintain multiple servers.

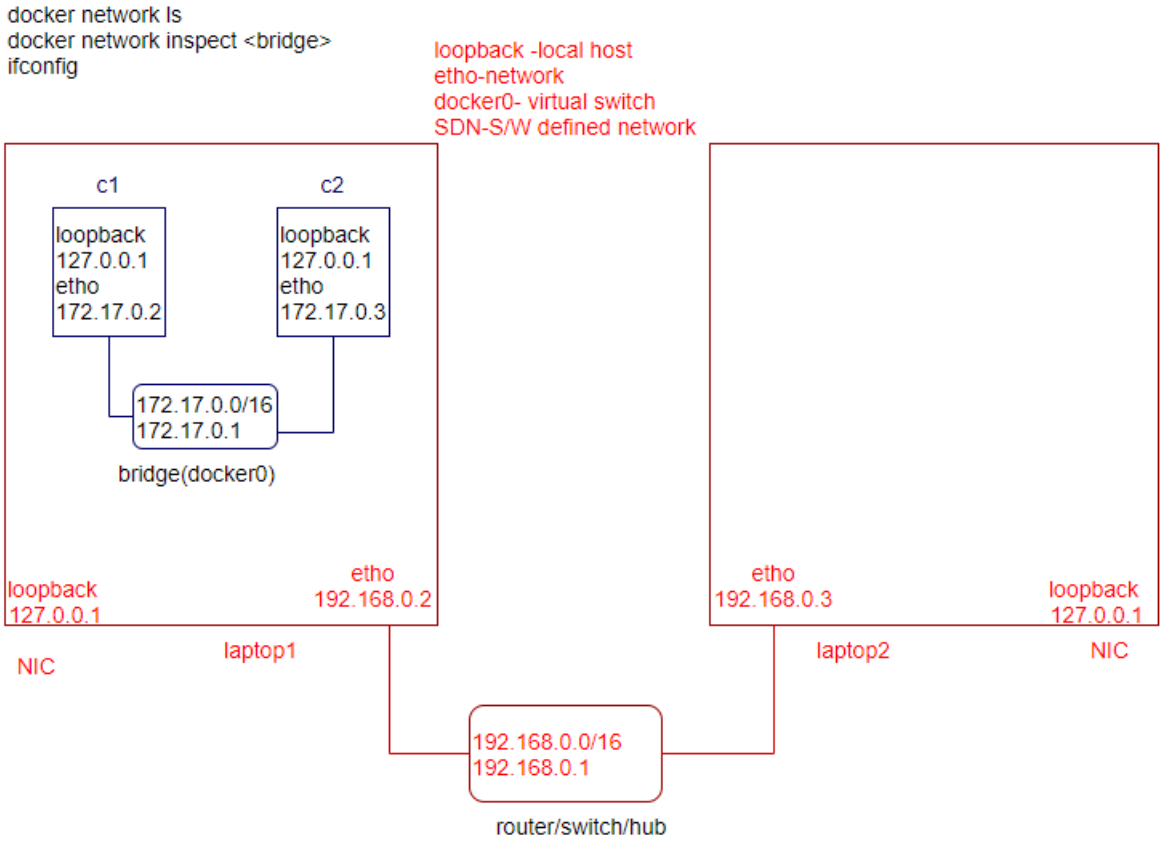
solution: container orchestration

- docker swarm
- kubernetes

Limitation: As of now we have discussed the networking within the same host, which is not a realistic scenario.

How containers communicate across different node?





By default, nodes are able to communicate to each other through the underlay network (AWS, GCP)

Docker Swarm

- **How to run multiple containers from given image into multiple vm/server/ec2?**
- **How to manage dynamic nature of the containers?**

Ans: We need an automation (Container Orchestration)

  - helps you to deploy the container into multiple servers
  - manage the containers across the multiple servers
  - provides various benefits.
    - Desired State
    - Scale-up/ Scale-down
    - request route to container (dynamic nature of container)

Cluster? Group of servers/ multiple servers

Container Orchestration tools?

- Docker Swarm (5%)
- Kubernetes(K8s)-95%

docker swarm cluster?

docker -v

docker info # swarm by default inactive

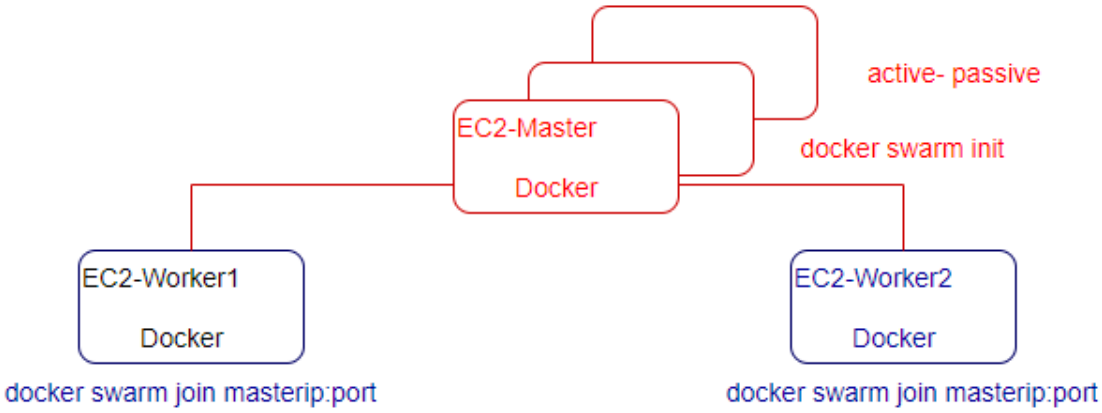
docker swarm init # vm/server act as a master

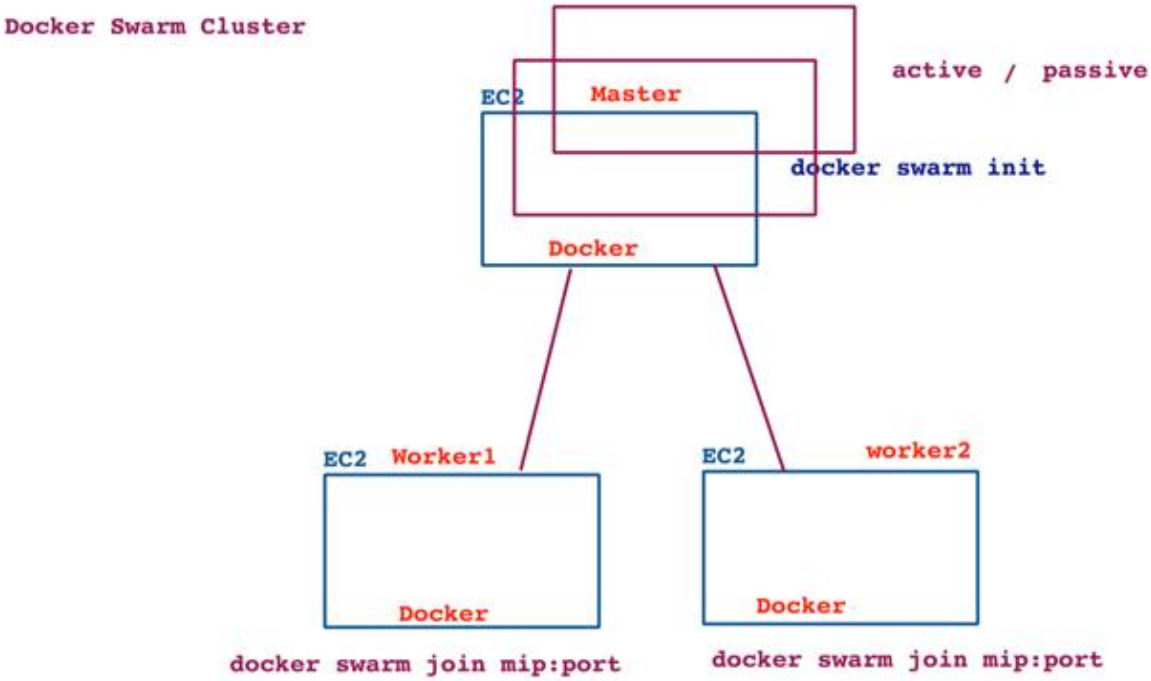
docker swarm join # worker node (vm/server will start reporting the master vm)

docker node ls # in master node

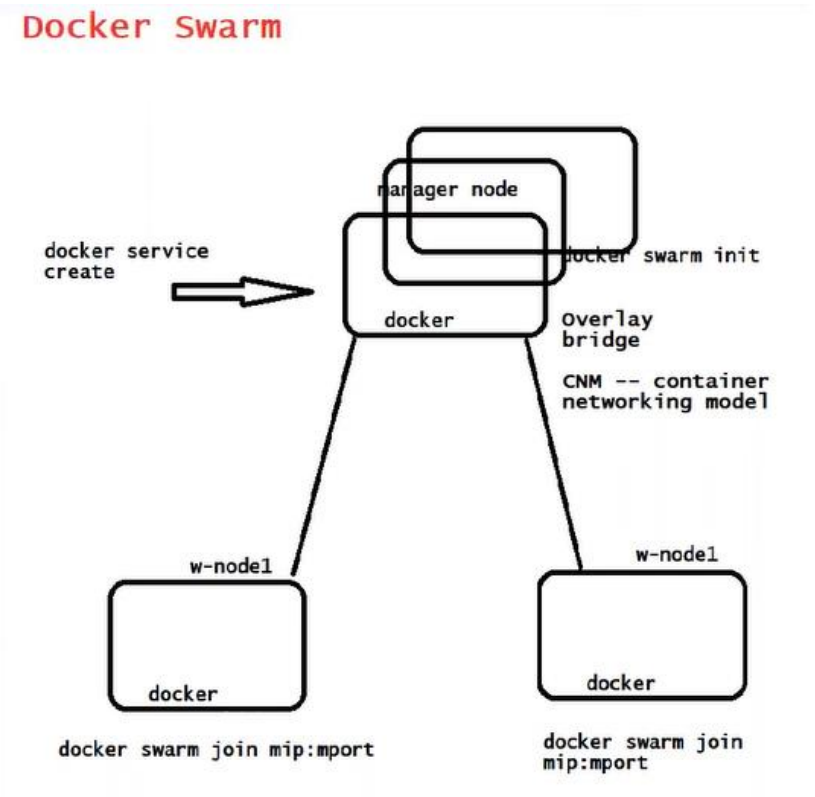
docker node inspect <node name>

docker node ps enode01





whereever we run "docker swarm init" that server will become or
will act as a docker swarm manager node/server



docker swarm capability is by default inbuilt with docker, which is by default inactive state, we activate the same using the command docker swarm init

```
root@controlplane:~# docker info | grep -i swarm
 Swarm: inactive
WARNING: No swap limit support
WARNING: No blkio weight support
WARNING: No blkio weight_device support
root@controlplane:~# docker network ls
NETWORK ID     NAME      DRIVER    SCOPE
d122251982d0   bridge    bridge    local
2cea1736596d   host      host      local
109b25cc349d   none      null      local
root@controlplane:~#
```

How to activate the swarm?

docker swarm init

Wherever we run “docker swarm init” that server become or will act as a docker swarm manager node/server.

As we initialize a docker swarm, docker creates a new extra network which span across the cluster, when you join the multiple docker hosts on the single cluster,

docker joins the multiple docker host in single virtual network again as an overlay network.

In master node

```
root@master:~# docker swarm init
Swarm initialized: current node (l4lhth287xo3maj6rpnysvze9) is now a manager.

To add a worker to this swarm, run the following command:

    docker swarm join --token SWMTKN-1-2j2jmg3w5ol9d1gjuwpmnl3i3ii7g4y520eahrbpm0g92my3f9-en4s597lrj76jo1pe1jidc78s 10.128.0.19:2377

To add a manager to this swarm, run 'docker swarm join-token manager' and follow the instructions.

root@master:~#
```

In worker node

```
root@node1:~# docker swarm join --token SWMTKN-1-2j2jmg3w5ol9d1gjuwpmnl3i3ii7g4y520eahrbpm0g92my3f9-en4s597lrj76jo1pe1jidc78s 10.128.0.19:2377
This node joined a swarm as a worker.
root@node1:~#
```

In master

docker node ls

```
root@master:~# docker node ls
ID                            HOSTNAME   STATUS    AVAILABILITY   MANAGER STATUS   ENGINE VERSION
l4lhth287xo3maj6rpnysvze9 *   master     Ready     Active         Leader           20.10.2
zddg99cmwvesse040ch1rp9v0     node1      Ready     Active                          20.10.2
38l497ht58u5iikcb2c4qr08f     node2      Ready     Active                          20.10.2
root@master:~#
```

How to deploy multiple containers together?

group of VMs-> cluster

group of containers-> service

How to deploy multiple containers (service)?

docker service create --name myapp –replicas=7 -p 9000:3000 lerndevops/samplepyapp:v1

docker service ls

docker service ps myapp # details cluster level command

docker ps # local command

docker service scale myapp=15

docker service scale myapp=2

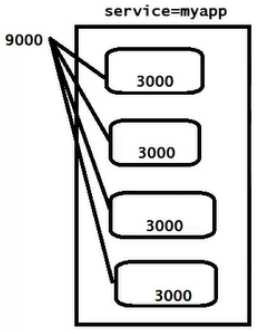
How to access container from browser

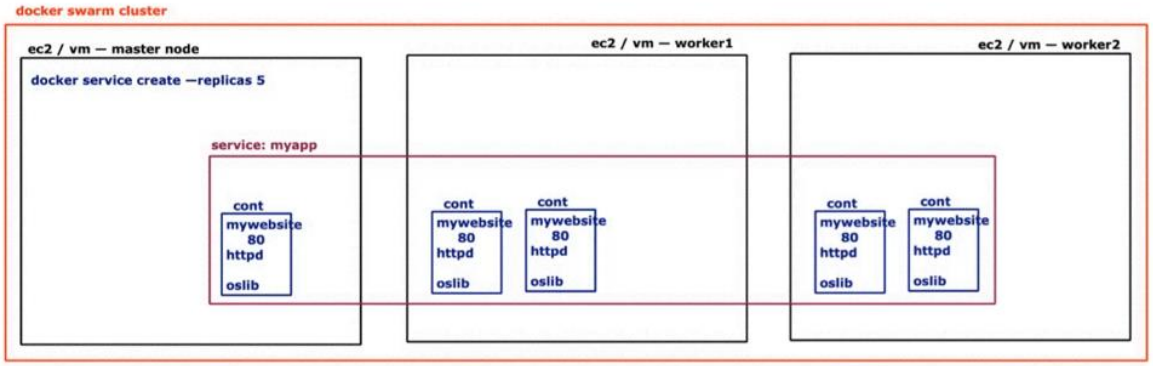
```
root@master:~# docker service ls
ID             NAME      MODE         REPLICAS   IMAGE                          PORTS
h0yzkoz2yqgz   myapp     replicated   4/4        lerndevops/samplepyapp:v2   *:9000->3000/tcp
root@master:~#
root@master:~# docker ps
CONTAINER ID   IMAGE                          COMMAND             CREATED          STATUS          PORTS      NAMES
2eaa068f2b17   lerndevops/samplepyapp:v2   "python myapp.py"   2 minutes ago   Up 2 minutes   3000/tcp   myapp.3.qszeyhhrzzfw4an0knd4wtpdz
bb24d426761f   lerndevops/samplepyapp:v2   "python myapp.py"   2 minutes ago   Up 2 minutes   3000/tcp   myapp.7.e3bz50cqvpe0i59ywpqj92wxr
root@master:~#
```

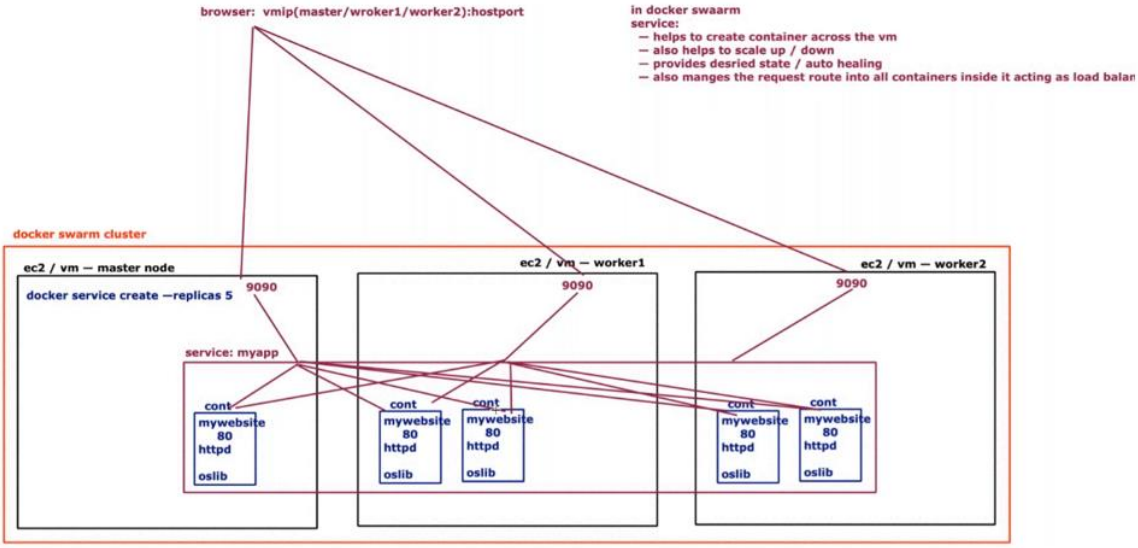
We expose the **port number for the service,** not for the container.

featuers of service in docker swarm

- can replicate number of reqyired cont
- scale up/down
- desired state
- acts as loadbalncer for forwarding the req into all backend cont







Features of service in docker-swarm

- Can replicate required number of containers
- Scaleup/down
- Cluster HA
- Auto healing (desired number of replicas)
- Act as a LB for forwarding the request into all backend containers.

docker service --help

docker service inspect <name>

docker service logs <name>

docker service rm <name>

docker service create --help

UC: How to create multiple service with single command

docker-compose (stack)

```
version: '3.7'
volumes:
  data:
  data-bkp:
services:
  springbootapp:
    image: lerndevops/springboot-mongo-app:latest
    deploy:
      replicas: 4
      placement:
        constraints:
          - node.labels.role==app
      restart_policy:
        condition: on-failure
      resources:
        limits:
          cpus: "0.1"
          memory: 150M
    ports:
      - "9090:8080"
    depends_on:
      - mongo
  mongo:
    image: lerndevops/mongo
    deploy:
      replicas: 1
      placement:
        constraints:
          - node.labels.role==db
      restart_policy:
        condition: on-failure
    ports:
      - "27017:27017"
    volumes:
      - data:/data/db
      - data-bkp:/data/bkp
```

UC: want to schedule container on specific node.

placement:

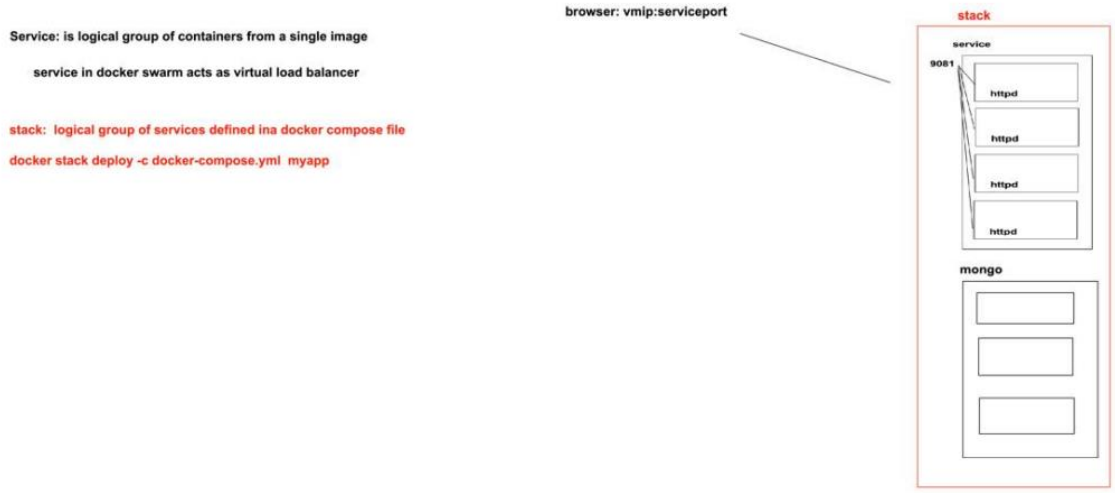
constraint:

node.labels.role==app

How to execute a docker compose file in docker swarm?

In case of docker-swarm, docker-compose creates stacks (group of services)

docker stack deploy -c mycompose. yaml <name of stack>



docker stack deploy -c mycompose.yaml myappstack

```
root@master:~# docker stack deploy  -c mycompose.yml myappstack
Creating service myappstack_springbootapp
failed to create service myappstack_springbootapp: Error response from daemon: invalid RestartCondition: "always"
root@master:~# vi mycompose.yml
root@master:~#
root@master:~#
root@master:~# docker stack deploy  -c mycompose.yml myappstack
Creating service myappstack_springbootapp
Creating service myappstack_mongo
root@master:~#
root@master:~# docker stack ls
NAME         SERVICES   ORCHESTRATOR
myappstack   2          Swarm
root@master:~# docker service ls
ID             NAME                      MODE         REPLICAS   IMAGE                                  PORTS
pn3rl71xg2dz   myappstack_mongo          replicated   0/1        lerndevops/mongo:latest                *:27017->27017/tcp
jr9hwit1zrkq   myappstack_springbootapp   replicated   0/4        lerndevops/springboot-mongo-app:latest   *:9090->8080/tcp
root@master:~#
```

Why services are not scheduled?

```
"Status": {
    "Timestamp": "2021-01-31T16:52:43.015627724Z",
    "State": "pending",
    "Message": "pending task scheduling",
    "Err": "no suitable node (scheduling constraints not satisfied on 3 nodes)",
    "PortStatus": {}
},
"DesiredState": "running",
```

How to update the node labels?

docker node update --label-add role=app node01

docker node update --label-add role=db node02

docker node inspect node01

docker node inspect node02

```
root@master:~# docker node update --label-add role=app node1
node1
root@master:~#
root@master:~# docker node update --label-add role=db node2
node2
root@master:~#
root@master:~#
```

```
root@master:~# docker service ls
ID             NAME                      MODE         REPLICAS   IMAGE                                      PORTS
pn3rl71xg2dz   myappstack_mongo          replicated   1/1        lerndevops/mongo:latest                    *:27017->27017/tcp
jr9hwit1zrkq   myappstack_springbootapp   replicated   4/4        lerndevops/springboot-mongo-app:latest     *:9090->8080/tcp
root@master:~#
root@master:~# docker service ps myappstack_springbootapp
ID             NAME                          IMAGE                                      NODE      DESIRED STATE   CURRENT STATE
   PORTS
p6tq2tg7rl5o   myappstack_springbootapp.1    lerndevops/springboot-mongo-app:latest     node1     Running         Running 55 seconds ago

yn6j6mj7wgx7   myappstack_springbootapp.2    lerndevops/springboot-mongo-app:latest     node1     Running         Running 55 seconds ago

427ocpvjkras   myappstack_springbootapp.3    lerndevops/springboot-mongo-app:latest     node1     Running         Running 54 seconds ago

af7m4n38rane   myappstack_springbootapp.4    lerndevops/springboot-mongo-app:latest     node1     Running         Running 55 seconds ago

root@master:~#
root@master:~#
root@master:~# docker service ps myappstack_mongo
ID             NAME                 IMAGE                      NODE      DESIRED STATE   CURRENT STATE            ERROR     PORTS
k2ku533mr6mj   myappstack_mongo.1   lerndevops/mongo:latest    node2     Running         Running 55 seconds ago
root@master:~#
```

```
root@master:~#
root@master:~# docker network ls
NETWORK ID     NAME                DRIVER    SCOPE
d2571fc7b614   bridge              bridge    local
17fb48182d61   docker_gwbridge     bridge    local
e38fc2b4994f   host                host      local
sz57qdr8zp8a   ingress             overlay   swarm
v9xqeud0k0eo   myappstack_default   overlay   swarm
c00874edb840   none                null      local
root@master:~#
```

Limitation:

- don’t provide autoscale.
- support only docker
- Container Network Model
- don’t support Helm
- RBAC (in-built security)

De-cluster docker swarm

1) delete all the services from master node

```
root@manager:~# docker service ls
ID             NAME                      MODE         REPLICAS   IMAGE                                    PORTS
kt5ucnserq3f   myappstack_mongo          replicated   1/1        lerndevops/mongo:latest                  *:27017->27017/
tcp
rczxw3favfcy   myappstack_springbootapp  replicated   4/4        lerndevops/springboot-mongo-app:latest   *:9090->8080/tc
p
acl13pqffklx   mysvc                     replicated   3/3        lerndevops/samplepyapp:v1                *:9080->3000/tc
p
root@manager:~# docker service rm kt5ucnserq3f rczxw3favfcy acl13pqffklx
kt5ucnserq3f
rczxw3favfcy
acl13pqffklx
root@manager:~#
```

2) docker swarm leave –f //in worker nodes.

```
root@w-node2:~# docker swarm leave -f
Node left the swarm.
root@w-node2:~#
```

First inactivate the docker swarm before installing K8s.

Difference between docker swarm and K8s?